Blockchain Experiment No 6

Aim: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Theory:
A Smart Contract is a self-executing program stored on the blockchain that automatically enforces the terms of an agreement when predefined conditions are met. It eliminates the need for intermediaries, ensuring transparency, security, and trust between parties. Smart contracts are written in Solidity (for Ethereum-based systems) and executed on the Ethereum Virtual Machine (EVM).

Significance of Smart Contracts in the Ethereum Blockchain:

● Automation: Executes actions automatically when conditions are satisfied. ●
Transparency: The contract code and transactions are visible on the blockchain. ●
Security: Once deployed, contracts are immutable and tamper-proof.
● Cost-effective: Removes intermediaries and reduces transaction costs. ● Trustless System: Participants can interact without needing to trust each other directly. Smart contracts form the backbone of decentralized applications (DApps) and decentralized finance (DeFi) systems on Ethereum.

Code:\

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract PharmaSupply {

   address public manufacturer;
   address public pharmacy;

   struct Medicine {
      string name;
      uint quantity;
      bool shipped;
      bool received;
   }

   Medicine public med;
```

```
    constructor(address _pharmacy) {
        manufacturer = msg.sender;
        pharmacy = _pharmacy;
    }

    function addMedicine(string memory _name, uint _quantity) public {
        require(msg.sender == manufacturer, "Only manufacturer
        allowed"); med = Medicine(_name, _quantity, false, false);
    }

    function shipMedicine() public {
        require(msg.sender == manufacturer, "Only manufacturer
        allowed"); med.shipped = true;
    }

    function receiveMedicine() public {
        require(msg.sender == pharmacy, "Only pharmacy allowed");
        require(med.shipped == true, "Not shipped yet");
        med.received = true;
    }

    function getMedicine() public view returns (string memory, uint, bool, bool)
        { return (med.name, med.quantity, med.shipped, med.received);
    }
}
```

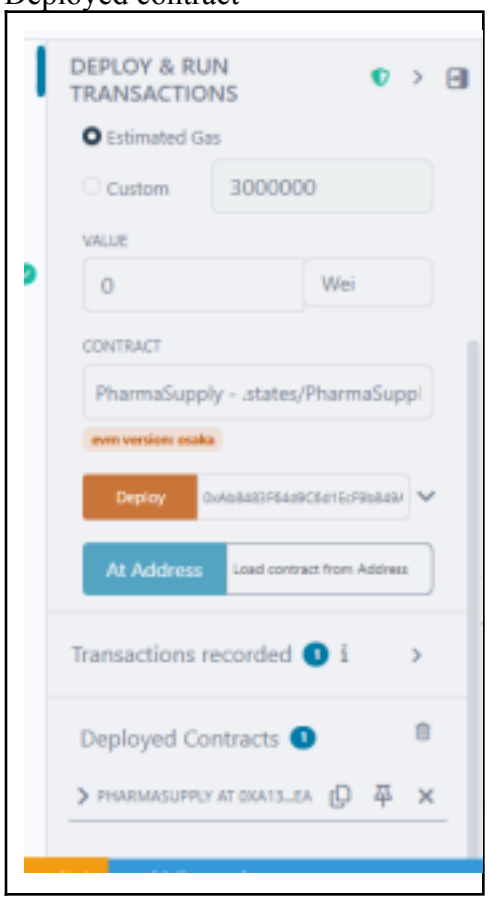
Implementation:

Contract 1: PharmaSupply (Medicine flow)

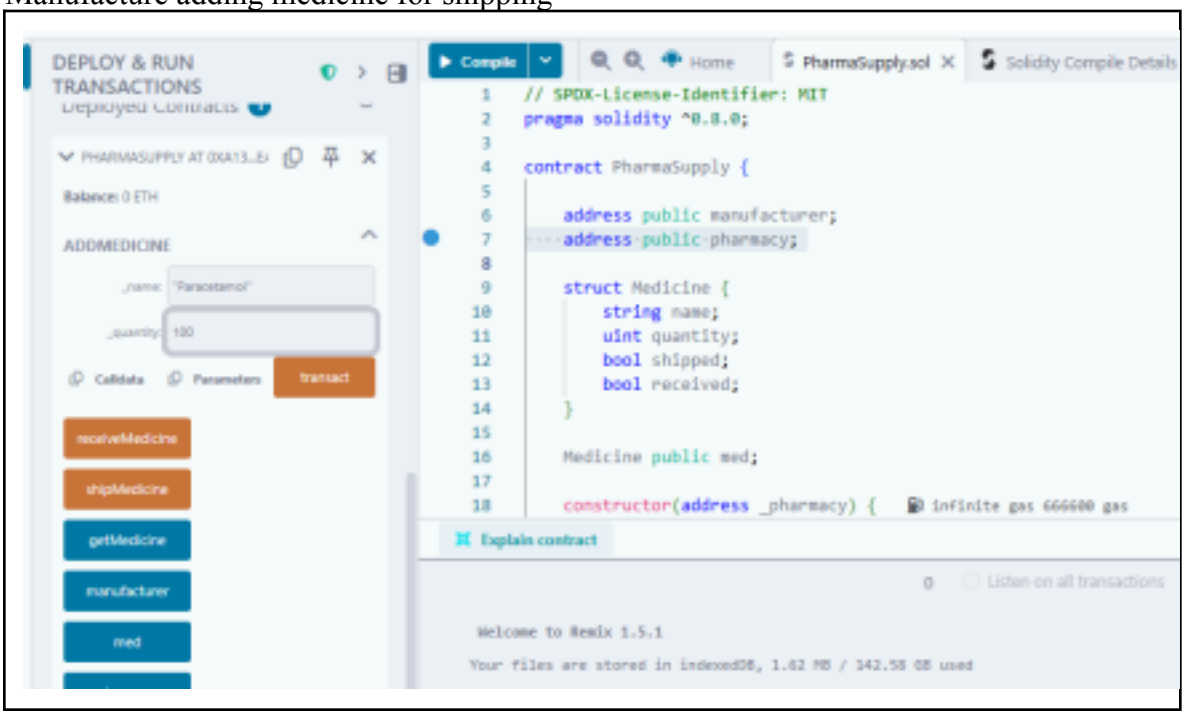
Contract code

Transaction deployed successfully

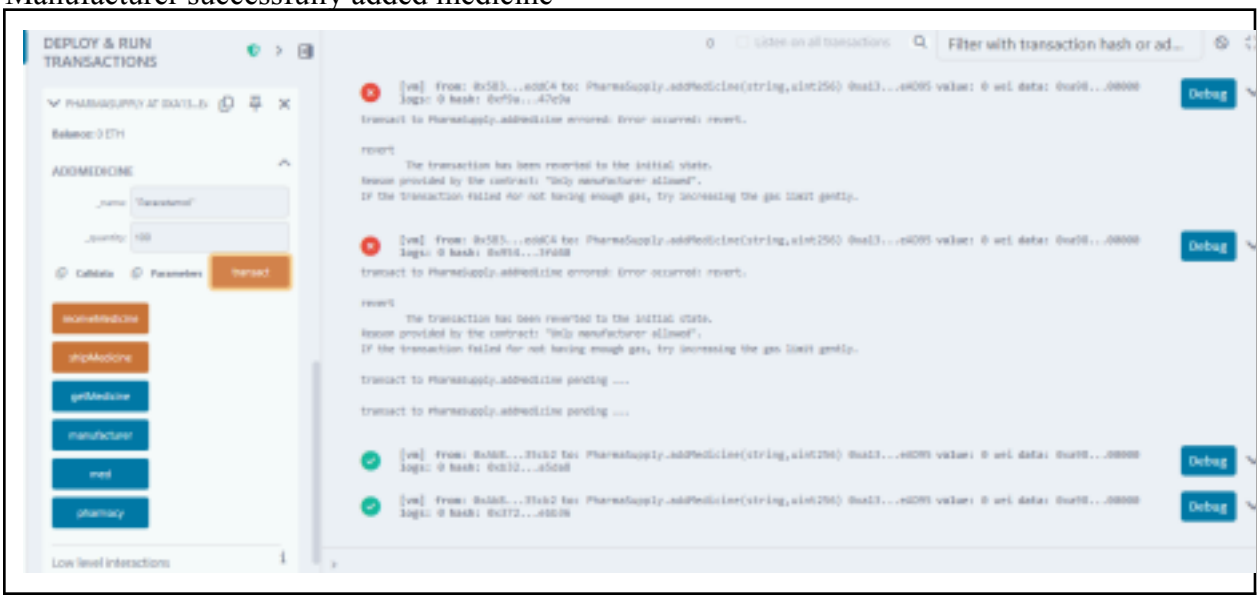
Deployed contract



Manufacture adding medicine for shipping



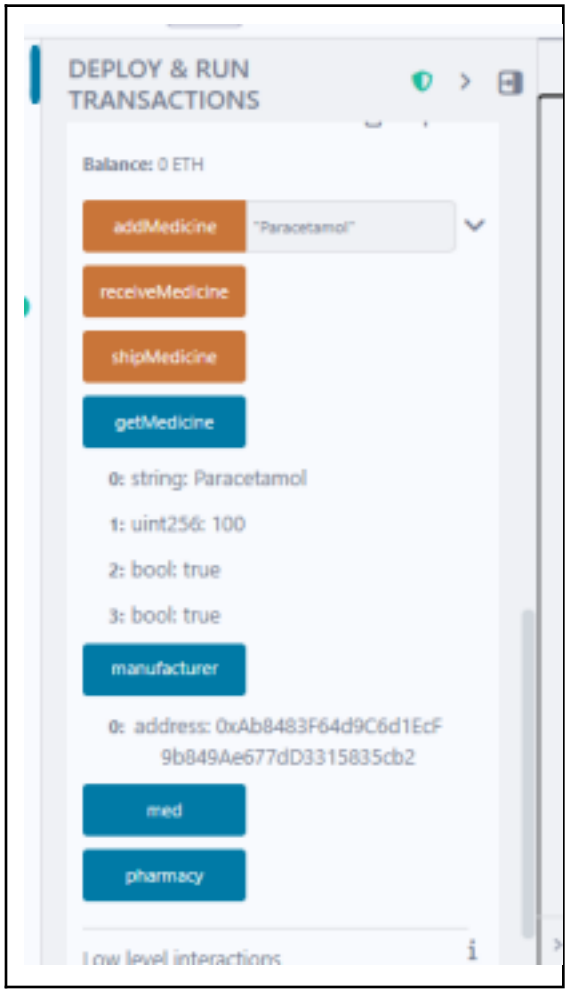
Manufacturer successfully added medicine



Manufacturer Shipping the medicine

Medicine received by pharmacy successfully

getmedicine() we are getting the values of added medicine



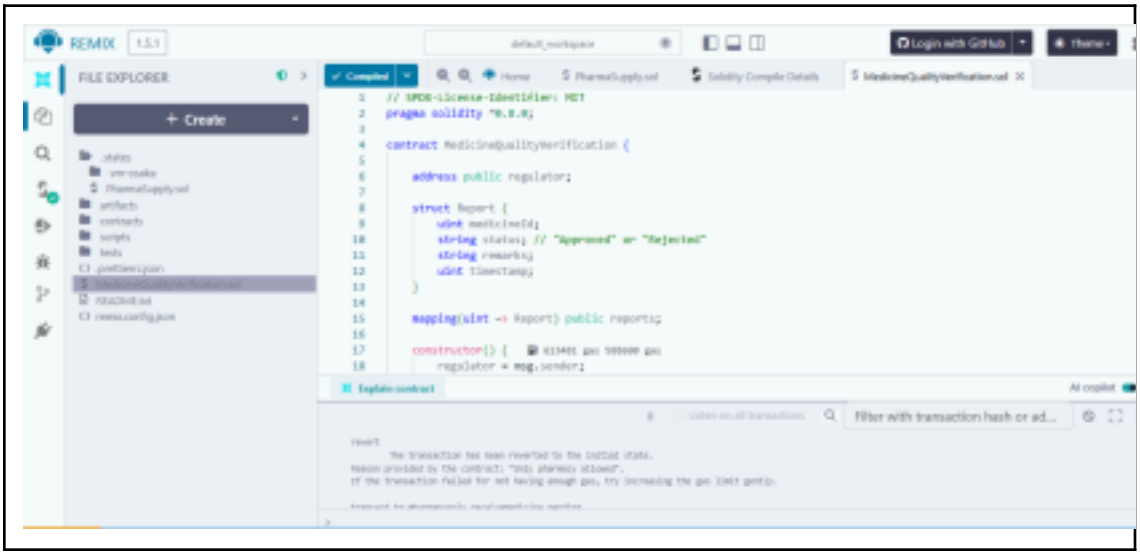
Contract 2: medicine quality verification

While verifying:

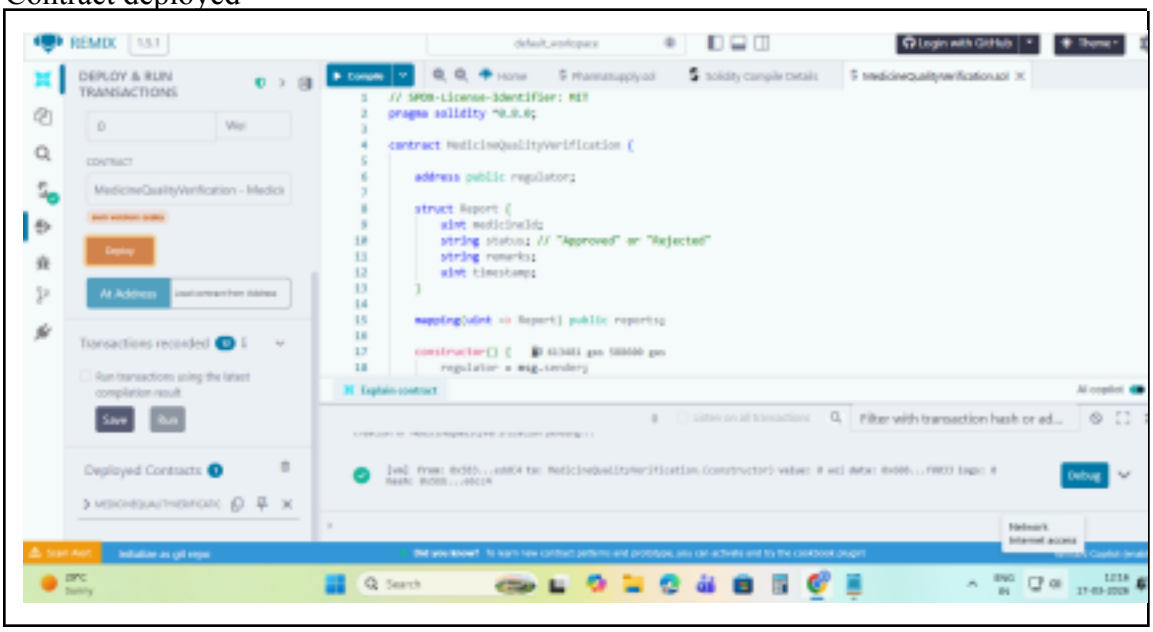
“Here, the regulator verifies the medicine batch and stores its approval status on blockchain.”

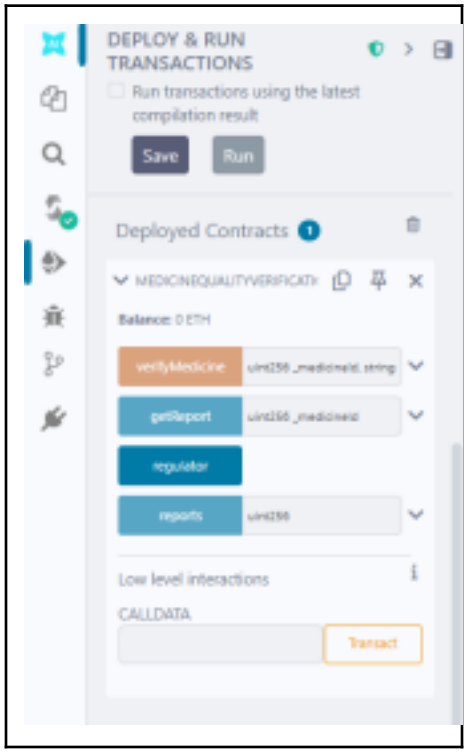
While fetching:

“This function retrieves the verification report, ensuring transparency and immutability.”

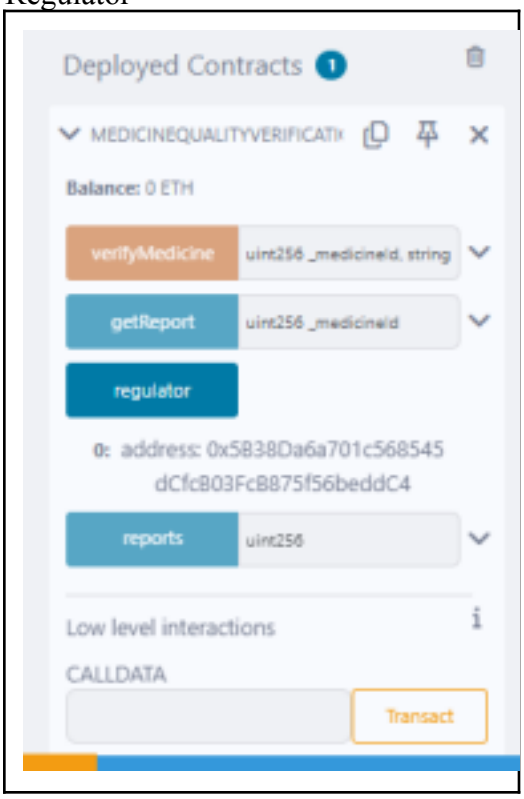


Contract deployed

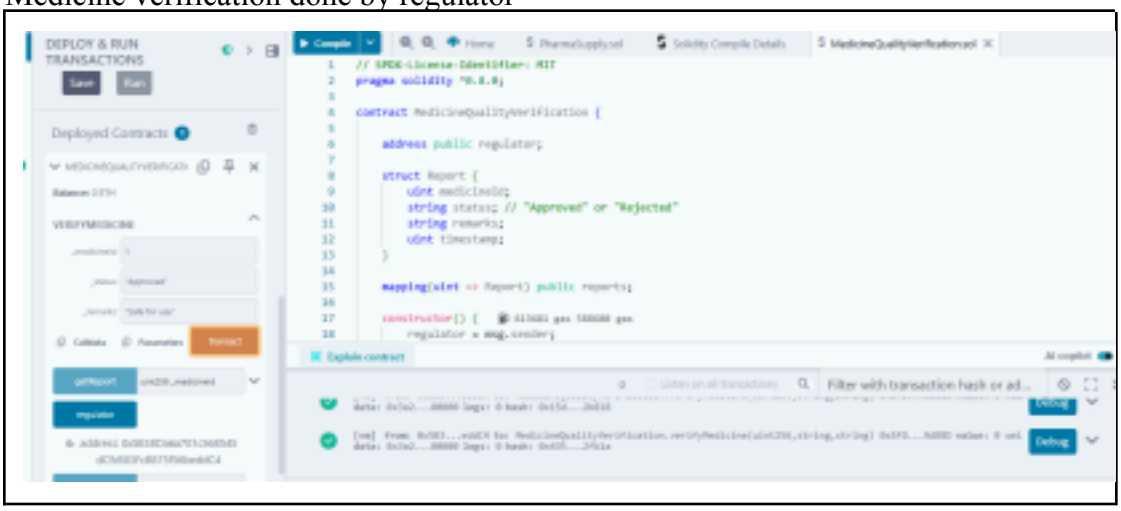



DEPLOY & RUN TRANSACTIONS
Run transactions using the latest compilation result
Save
Run
Deployed Contracts 1
MEDICINEQUALITYVERIFICATI
Balance: 0 ETH
verifyMedicine
uint256 _medicineId, string
getReport
uint256 _medicineId
regulator
reports
uint256
Low level interactions
CALLDATA
Transact

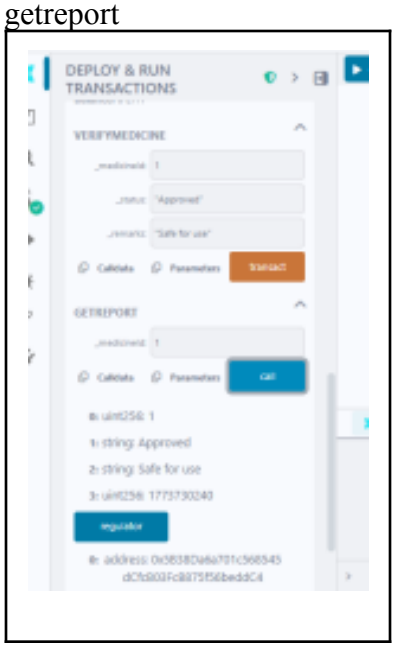
Regulator


Deployed Contracts 1
MEDICINEQUALITYVERIFICATI
Balance: 0 ETH
verifyMedicine
uint256 _medicineId, string
getReport
uint256 _medicineId
regulator
0: address: 0x5838Da6a701c568545dCfcB03FcB875f56beddC4
reports
uint256
Low level interactions
CALLDATA
Transact

Medicine verification done by regulator



getreport



Conclusion:

The experiment was successfully completed using Remix IDE, where two Solidity smart contracts were deployed and tested on the Ethereum blockchain. Contract 1 (PharmaSupply) demonstrated the medicine flow from manufacturer to pharmacy, while Contract 2 (MedicineQualityVerification) allowed a regulator to verify and store quality reports on-chain. Role-based access control was enforced using require statements, and all transactions were recorded immutably. The experiment successfully demonstrated the use of smart contracts for transparent and trustless pharmaceutical supply chain management.